

北京邮电大学

智能交互机器人实验



题目： YouGuide | 邮小导——面向北京邮电大学
高校迎新场景的多机器人协同导航与智能交互系统

学 号： 2022211733

姓 名： 王殿云

学科专业： 人工智能

班 级： 2022219107

学 院： 人工智能学院

2025 年 6 月 15 日

摘 要

YouGuide 系统是一个面向北京邮电大学高校迎新场景的多机器人协同导航与智能交互解决方案。针对传统迎新模式中人力资源紧张、信息管理复杂、空间导航困难、服务连续性不足等痛点问题，本项目创新性地提出了基于多机器人协同的智能化迎新服务体系。

系统采用分布式架构设计，通过机器人子系统、智能体子系统、导航子系统和管理子系统的有机整合，实现了自然语言交互、智能问答、精准导航、个性化服务等核心功能。系统集成大语言模型（LLM）、ReAct Agent 框架、RAG 检索增强生成、实时语音识别与合成、人脸识别、GPS 导航等前沿技术，构建了完整的“感知-理解-决策-执行”智能服务闭环。

在技术创新方面，系统实现了引导机器人与答疑机器人的角色分工与动态切换、基于任务调度的多机器人协同导航、流式语音交互的低延迟响应、知识驱动的智能问答服务等特色功能。通过在实验室环境下的模拟验证，YouGuide 系统展示了提升迎新工作效率的潜力，为高校智慧校园建设提供了可落地、可推广的创新范例。

目 录

1. 前言	5
1.1 项目背景	5
1.2 主要工作与贡献	5
1.3 项目核心创新点	5
2. 业务背景与现状分析	6
2.1 业务背景	6
2.2 现状分析	6
2.2.1 人力资源紧张	6
2.2.2 信息管理复杂	6
2.2.3 空间导航困难	7
2.2.4 服务连续性不足	7
3. 需求分析	7
3.1 智能交互服务需求	8
3.2 精准导航引导需求	8
3.3 个性化服务需求	8
3.4 协同工作需求	8
3.5 知识管理需求	8
3.6 管理监控需求	8
4. 系统设计	9
4.1 系统角色设计	9
4.1.1 新生	9
4.1.2 志愿者	10
4.1.3 系统管理员	10
4.1.4 迎新机器人	11
4.2 系统架构设计	14
4.2.1 系统设计原则	14
4.2.2 系统计算资源分配	14
4.2.3 系统总架构与子系统划分	15
4.2.4 系统技术选型	16
4.3 子系统设计	17
4.3.1 机器人子系统设计	17
4.3.2 智能体系统设计	22
4.3.3 导航系统设计	27
4.3.4 管理系统设计	32
5. 实验平台说明	44
5.1 硬件平台	44

5.2 软件与开发环境	45
5.3 平台限制说明	46
6. 总结与展望	46
6.1 项目成果总结	46
6.2 存在的不足与改进方向	47

1. 前言

1.1 项目背景

随着人工智能技术的快速发展和高等教育规模的不断扩大，传统的高校管理模式正面临着前所未有的挑战与机遇。北京邮电大学作为信息科技领域的重要高等学府，每年迎接数千名新生入学，迎新工作的质量直接影响着学生的第一印象和归属感。

然而，现有的迎新模式主要依赖人工服务，在实际运行中暴露出诸多问题：志愿者数量有限导致服务能力不足、信息传递准确性难以保证、新生在陌生环境中容易迷路、服务的连续性和个性化程度较低。这些问题不仅增加了学校的管理负担，也影响了新生的报到体验。

在此背景下，如何运用人工智能、机器人技术等现代科技手段，构建智能化、人性化的迎新服务体系，成为亟待解决的重要课题。YouGuide 项目正是在这样的背景下应运而生，旨在通过技术创新推动高校管理服务的智能化转型。

1.2 主要工作与贡献

本项目的主要工作包括：

1. **需求分析与系统设计**：深入调研北京邮电大学迎新场景的实际需求，设计了涵盖智能交互、精准导航、个性化服务、协同工作等功能的完整系统架构。
2. **关键技术研发**：
 - 实现了基于 ReAct 的智能体决策框架，使机器人具备自主推理和工具调用能力
 - 开发了流式语音交互系统，大幅降低了人机对话的延迟
 - 构建了多机器人协同导航算法，实现了任务的智能分配和路径优化
 - 集成了 RAG 知识检索系统，确保信息服务的准确性和时效性
3. **系统实现与集成**：基于 ROS、Dify、Spring Boot 等成熟框架，实现了从感知层到应用层的完整系统，并通过标准化接口实现了各子系统的无缝集成。

1.3 项目核心创新点

YouGuide 系统在以下方面实现了创新突破：

1. **多机器人角色协同机制**：首创了引导型和答疑型机器人的动态角色分工模式，通过远程控制实现角色切换，大幅提升了服务灵活性。
2. **端云协同的分布式架构**：创新性地将计算任务在机器人端、近端计算端、远端服

务端进行合理分配，既保证了实时响应，又充分利用了云端算力。

3. **知识驱动的智能交互**：通过 RAG 技术与 ReAct 框架的深度融合，实现了基于校园知识库的精准问答服务，有效避免了大模型的幻觉问题。

4. **情境感知的个性化服务**：结合人脸识别、位置追踪、进度管理等技术，为每位新生提供量身定制的迎新服务。

5. **虚实融合的导航体系**：将数字地图导航与实体机器人引导相结合，提供了全方位的空间导航解决方案。

2. 业务背景与现状分析

2.1 业务背景

北京邮电大学沙河校区每年迎接数千名本科新生入学，迎新工作是高校管理中的重要环节。当前的迎新流程涉及多个环节，包括入校办理、资料领取、宿舍入住等步骤，新生需要在陌生的校园环境中前往多个地点完成报到手续。整个报到流程复杂且涉及多个部门协同，对初次踏入校园的新生来说存在较大的挑战。



传统的迎新模式主要依靠人工服务，由志愿者和工作人员为新生提供咨询、引导等服务。然而，这种模式在实际运行中暴露出诸多问题，亟需通过智能化手段进行改进和优化。

2.2 现状分析

2.2.1 人力资源紧张

迎新期间，大量新生集中报到造成了巨大的服务压力。志愿者数量有限，难以满足高峰期的服务需求。每位志愿者需要同时应对多名新生的咨询，经常出现排队等待的情况。此外，志愿者需要长时间工作，疲劳状态下服务质量难以保证。人力资源的局限性直接影响了迎新工作的效率和新生的报到体验。

2.2.2 信息管理复杂

迎新涉及的文档资料繁多，包括报到流程说明、各部门规章制度、校园生活指南等。这些信息分散在不同的文件中，新生难以快速找到所需信息。志愿者在回答问题时也需要查阅大量资料，影响了响应速度。同时，口头传达信息容易出现遗漏或误解，信息传递的准确性和完整性难以保证。

本科生迎新通知		➔ 更多
• 人工智能入门课《AI在大前》开课通知	[08-15]	
• 关于组织2024级本科新生入学英语分级测试的通知	[08-09]	
• 2024年北京邮电大学迎新系统说明（本科生）	[07-09]	
• 关于新生入学英语分级测试有关问题的集中解答	[08-17]	
• 北京邮电大学沙河校区平面图	[08-09]	

北京邮电大学迎新网上部分文档

2.2.3 空间导航困难

沙河校区占地面积大，建筑物分布广泛，新生对校园环境完全陌生。报到流程需要在多个地点之间往返，如从教学实验综合楼到图书馆，再到雁北园或雁南园宿舍区。新生经常出现迷路、走错地点的情况，不仅延长了报到时间，也增加了焦虑情绪。即使有校园地图，新生在陌生环境中仍难以快速定位和规划路线。



2.2.4 服务连续性不足

现有的人工服务模式，新生在不同地点遇到的是不同的工作人员，每次咨询都需要重新说明自己的情况和需求。工作人员之间信息共享不足，无法为新生提供连贯的个性化服务。新生的报到进度、已完成事项等信息难以实时跟踪，影响了服务的针对性和效率。

3. 需求分析

基于上节分析得到的业务背景，北京邮电大学迎新场景对智能交互服务机器人系

统有以下核心需求：

3.1 智能交互服务需求

系统需要提供自然、高效的人机交互能力，支持语音、图像等多模态输入，让新生能够以最自然的方式表达需求。机器人应具备智能理解和推理能力，能够准确识别新生的意图，并给出恰当的响应。通过集成先进的自然语言处理技术和知识图谱，机器人能够回答各类迎新相关问题，从基础的流程咨询到复杂的政策解读，都能提供准确、详细的解答。

3.2 精准导航引导需求

系统需要解决新生在陌生校园环境中的导航难题。机器人应具备自主导航能力，能够规划最优路径并引导新生前往目标地点。通过实体机器人的跟随式引导，新生可以轻松到达各个报到点，避免迷路带来的时间浪费和心理压力。导航功能应覆盖整个报到流程涉及的所有地点，包括各学院办公台、宿舍楼、图书馆等关键位置。

3.3 个性化服务需求

系统需要为每位新生提供个性化的服务体验。通过人脸识别等技术手段，机器人能够识别新生身份，获取其专业、宿舍分配、报到进度等个人信息。基于这些信息，机器人可以提供定制化的服务建议，如根据新生当前的报到阶段推荐下一步应该前往的地点，或者根据专业特点提供相关的入学指导。这种个性化服务能够大大提升迎新效率和新生满意度。

3.4 协同工作需求

面对大规模的迎新服务需求，单一机器人难以应对。系统需要支持多机器人协同工作，通过合理的任务分配和角色定位，实现服务能力的倍增。引导型机器人专注于导航服务，答疑型机器人专注于信息咨询，两者相互配合，形成完整的服务链条。同时，系统应支持机器人之间的信息共享和任务转接，确保服务的连续性和一致性。

3.5 知识管理需求

系统需要建立完善的知识管理机制，将分散的迎新文档、政策规定、常见问题等信息进行结构化整理和存储。通过检索增强生成（RAG）等技术，机器人能够快速准确地获取相关信息，减少信息传递中的错误和遗漏。知识库应支持便捷更新，确保信息的时效性和准确性。

3.6 管理监控需求

系统需要提供完善的后台管理功能，让管理人员能够实时监控机器人运行状态、服务质量和新生报到进度。通过数据统计和分析，管理者可以及时发现服务瓶颈，优化资源配置。系统还应支持远程控制和干预，在特殊情况下由人工接管服务，确保迎新工作的顺利进行。

4. 系统设计

4.1 系统角色设计

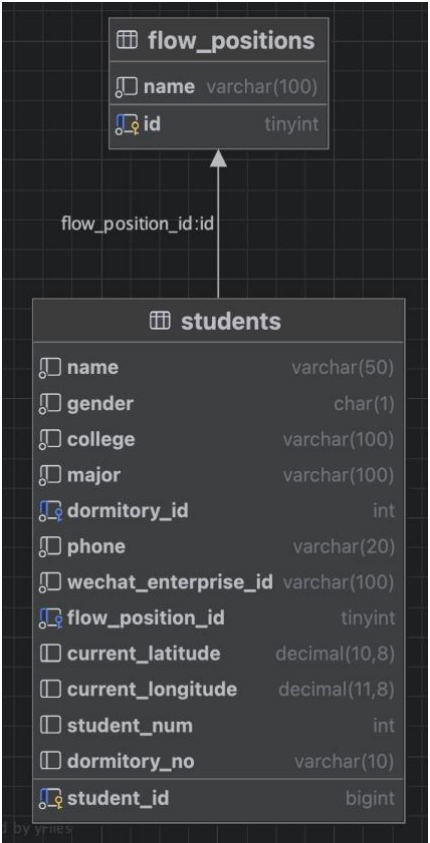
4.1.1 新生

4.1.1.1 描述

新生是系统的核心服务对象，系统通过对入校新生建立完整的数据模型，跟踪其从入校到完成所有报道流程的全过程。系统支持新生信息的预录入，并通过流程进度字段实时跟踪每位新生的报道状态，确保报道流程的有序进行。

4.1.1.2 数据字段

- **student_id** (bigint): 学生唯一标识，系统自动生成
- **student_num** (int): 学号，用于身份验证和信息查询
- **name** (varchar(50)): 姓名
- **gender** (char(1)): 性别（男/女）
- **college** (varchar(100)): 所属学院
- **major** (varchar(100)): 专业
- **dormitory_id** (int): 宿舍楼编号，关联到具体的宿舍楼位置
- **dormitory_no** (varchar(10)): 宿舍房间号
- **phone** (varchar(20)): 联系电话
- **wechat_enterprise_id** (varchar(100)): 微信企业号 ID，用于系



统通知和身份绑定

- **flow_position_id** (tinyint): 报道流程进度, 包含四个阶段:

- 0: 尚未入校
- 1: 办理入校
- 2: 办理领取资料
- 3: 办理宿舍入住

- **current_latitude** (decimal(10,8)): 当前位置纬度

- **current_longitude** (decimal(11,8)): 当前位置经度

4.1.2 志愿者

4.1.2.1 描述

志愿者是迎新过程中的重要人力资源, 与机器人协同工作, 为新生提供必要的人工服务支持。系统通过实时定位功能, 帮助管理员合理调度志愿者资源, 实现人机协同的最优配置。

4.1.2.2 数据字段

- **volunteer_id** (int): 志愿者唯一标识, 系统自动生成

- **name** (varchar(50)): 姓名

- **gender** (char(1)): 性别 (男/女)

- **current_latitude** (decimal(10,8)): 当前位置纬度

- **current_longitude** (decimal(11,8)): 当前位置经度

volunteers	
name	varchar(50)
gender	char(1)
current_latitude	decimal(10,8)
current_longitude	decimal(11,8)
volunteer_id	int

4.1.3 系统管理员

4.1.3.1 描述

系统管理员负责整个迎新系统的运维和管理，包括新生信息的录入与维护、志愿者的调度管理、机器人的状态监控与远程控制等。系统采用基于角色的权限管理(RBAC)，确保管理功能的安全性和可控性。

4.1.3.2 数据字段

- **user_id** (bigint): 用户唯一标识
- **dept_id** (bigint): 所属部门 ID
- **user_name** (varchar(30)): 登录账号
- **nick_name** (varchar(30)): 显示昵称
- **user_type** (varchar(2)): 用户类型 (00 表示系统用户)
- **email** (varchar(50)): 电子邮箱
- **phonenummer** (varchar(11)): 手机号码
- **sex** (char(1)): 性别 (0 男 1 女 2 未知)
- **avatar** (varchar(100)): 头像地址
- **password** (varchar(100)): 加密后的密码
- **status** (char(1)): 账号状态 (0 正常 1 停用)
- **login_ip** (varchar(128)): 最后登录 IP
- **login_date** (datetime): 最后登录时间
- **create_time** (datetime): 创建时间
- **update_time** (datetime): 更新时间
- **remark** (varchar(500)): 备注信息



The image shows a screenshot of a database table structure for a table named `sys_user`. The table has the following fields and data types:

Field	Data Type
<code>dept_id</code>	<code>bigint</code>
<code>user_name</code>	<code>varchar(30)</code>
<code>nick_name</code>	<code>varchar(30)</code>
<code>user_type</code>	<code>varchar(2)</code>
<code>email</code>	<code>varchar(50)</code>
<code>phonenummer</code>	<code>varchar(11)</code>
<code>sex</code>	<code>char(1)</code>
<code>avatar</code>	<code>varchar(100)</code>
<code>password</code>	<code>varchar(100)</code>
<code>status</code>	<code>char(1)</code>
<code>del_flag</code>	<code>char(1)</code>
<code>login_ip</code>	<code>varchar(128)</code>
<code>login_date</code>	<code>datetime</code>
<code>create_by</code>	<code>varchar(64)</code>
<code>create_time</code>	<code>datetime</code>
<code>update_by</code>	<code>varchar(64)</code>
<code>update_time</code>	<code>datetime</code>
<code>remark</code>	<code>varchar(500)</code>
<code>user_id</code>	<code>bigint</code>

4.1.4 迎新机器人

4.1.4.1 描述

迎新机器人是本系统的核心创新点，通过引导机器人和答疑机器人两种角色的协同工作，为新生提供智能化的导航和咨询服务。机器人具备自主导航、人机交互、角色切换等能力，能够根据新生的实际需求提供个性化服务。

迎新机器人主要分为引导机器人与答疑机器人两类，两类机器人功能有所重叠，但主要负责事务不同。引导机器人着重设计导航，通过跟随学生的方式来引导学生前往对应的报道点位。答疑机器人着重设计交互，通过更好的交互答疑来帮助学生解疑答惑。两类机器人在系统的协作下共同工作，同时可以远程控制以互相转换角色，引导学生完成报道流程。

4.1.4.2 引导机器人

4.1.4.2.1 描述

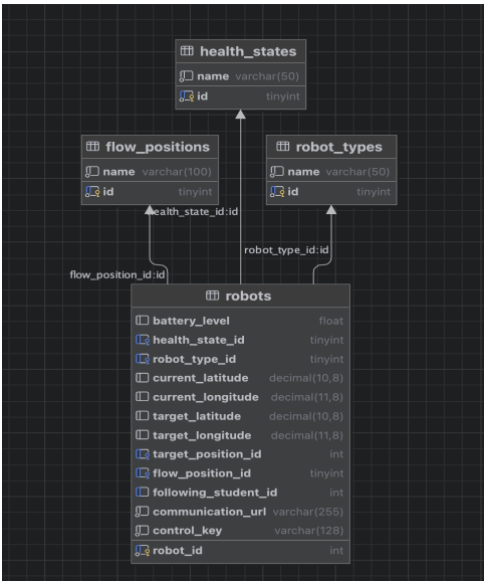
引导机器人专注于物理导航功能，通过 GPS 定位和路径规划算法，能够准确地将新生引导至目标地点。机器人采用跟随模式，确保新生在陌生环境中不会迷路，并能实时调整路线以应对突发情况。

4.1.4.2.2 动作空间

- 移动导航：前进、后退、转向、停止
- 交互动作：挥手、点头、转身指向
- 状态切换：在远程控制下切换至答疑模式、进入待机状态

4.1.4.2.3 数据字段

- **robot_id** (int): 机器人唯一标识
- **robot_type_id** (tinyint): 机器人类型 (0:引导机器人)
- **battery_level** (float): 当前电量百分比
- **health_state_id** (tinyint): 健康状态
 - 0: 不在线
 - 1: 在线
 - 2: 低电量
- **current_latitude** (decimal(10,8)): 当前位置纬度
- **current_longitude**



(decimal(11,8)): 当前位置经度

- **target_latitude** (decimal(10,8)): 目标位置纬度
- **target_longitude** (decimal(11,8)): 目标位置经度
- **target_position_id** (int): 目标地点 ID (关联到预设的关键位置)
- **following_student_id** (int): 当前跟随的新生 ID
- **flow_position_id** (tinyint): 当前服务的流程阶段
- **communication_url** (varchar(255)): 通信接口地址
- **control_key** (varchar(128)): 远程控制密钥

4.1.4.3 答疑机器人

4.1.4.3.1 描述

答疑机器人专注于信息交互和咨询服务，集成了先进的自然语言处理能力和知识检索系统。通过 ReAct Agent 框架，机器人能够理解新生的各类问题，并通过调用相应的工具提供准确、及时的答复。

4.1.4.3.2 动作空间

- 语音交互：语音识别、语音合成、多轮对话
- 视觉交互：人脸识别、场景理解
- 动作交互：单步移动、点头、手势指引
- 信息检索：RAG 文档查询、实时信息获取
- 任务调度：为引导机器人分配任务、协调多机器人协作

4.1.4.3.3 数据字段

- **robot_id** (int): 机器人唯一标识
- **robot_type_id** (tinyint): 机器人类型 (1:答疑机器人)
- **battery_level** (float): 当前电量百分比
- **health_state_id** (tinyint): 健康状态
 - 0: 不在线
 - 1: 在线
 - 2: 低电量

- **current_latitude** (decimal(10,8)): 当前位置纬度
- **current_longitude** (decimal(11,8)): 当前位置经度
- **communication_url** (varchar(255)): 通信接口地址
- **control_key** (varchar(128)): 远程控制密钥

4.2 系统架构设计

4.2.1 系统设计原则

YouGuide 系统采用分布式架构设计理念，充分考虑了高校迎新场景下的特殊需求与约束条件。系统设计遵循三个核心原则：首先是端侧最小计算原则，考虑到机器人端计算资源的有限性，系统将计算密集型任务合理分配到不同计算节点，确保实时性要求高的任务在近端处理，而复杂计算则通过 API 调用远端服务完成。其次是模块高度解耦原则，系统采用微服务架构思想，各功能模块之间通过明确定义的接口进行通信，降低了系统的耦合度，提高了可维护性和可扩展性。最后是分布式设计原则，管理端采用前后端分离架构，各子系统间优先使用网络通信协议，通过 RESTful API 和 WebSocket 等标准化接口实现数据交换，避免了传统的紧耦合设计带来的维护困难。

4.2.2 系统计算资源分配

4.2.2.1 机器人端

机器人端作为系统的执行终端，主要负责基础的传感器数据采集、运动控制和本地响应处理。为了保证实时性和降低网络依赖，机器人端运行精简的 ROS 节点，处理音频流采集、图像采集、基础运动控制等任务。通过 Flask 服务器对外暴露标准化的 HTTP 接口，使得上层应用能够便捷地调用机器人的动作执行和传感器数据获取功能。这种设计确保了即使在网络不稳定的情况下，机器人仍能维持基本的运动和感知能力。

4.2.2.2 近端侧计算端

近端侧计算端承担了系统的核心智能处理任务，主要运行 Agent 决策逻辑和工具调用协调。该层通过 Dify 框架整合了 ReAct Agent 的推理能力，实现了工具调用的智能编排。当用户与机器人交互时，近端计算端负责理解用户意图、规划行动序列、调用相应工具并整合执行结果。这种设计既保证了交互的流畅性，又避免了将所有智能计算压力集中在机器人端，实现了计算资源的合理分配。

4.2.2.3 远端侧管理端

远端管理端作为系统的中枢控制中心，负责全局状态管理、数据持久化和系统监控。基于 Spring Boot 框架构建的管理后台提供了完整的机器人状态监控、任务调度、

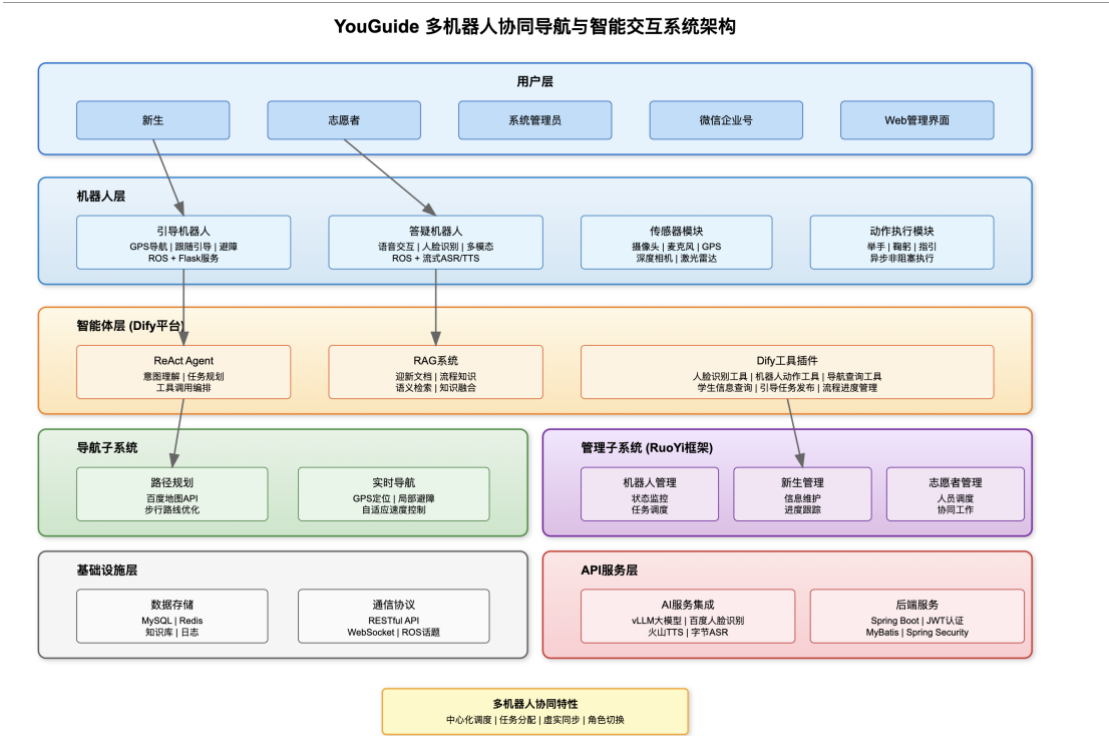
用户管理等功能。通过 MySQL 数据库存储系统运行数据，Redis 缓存提供高速数据访问，确保了系统在高并发场景下的稳定运行。管理端还负责协调多个机器人之间的任务分配，实现了真正意义上的多机器人协同工作。

4.2.2.4 远端侧 API 服务端

API 服务端整合了系统所需的各类 AI 能力服务，包括基于 vLLM 的大语言模型推理服务、百度 AI 的人脸识别服务、火山引擎的流式语音合成服务以及字节跳动的流式语音识别服务。通过统一的 API 网关对外提供服务，屏蔽了不同服务提供商的接口差异，为上层应用提供了标准化的调用方式。这种设计不仅提高了系统的可扩展性，还便于后续更换或升级各类 AI 服务。

4.2.3 系统总架构与子系统划分

YouGuide 系统采用四层架构设计，自下而上分别为机器人子系统、智能体子系统、导航子系统和系统管理子系统。



4.2.3.1 机器人子系统

机器人子系统基于 ROS 框架构建，通过标准的 ROS 节点实现音频采集、语音合成、图像处理等基础功能，并通过 Flask 服务器封装动作执行和人脸识别等高级功能，对外提供 RESTful API 接口。

4.2.3.2 智能体子系统

智能体子系统是系统的智能核心，采用 Dify 平台作为 Agent 运行时环境，集成了 RAG 检索增强生成能力和 ReAct 推理框架。系统创新性地开发了一系列 Dify 插件工具，包括人脸识别与学生信息查询工具、机器人动作执行工具、导航路线查询工具和引导任务发布工具，这些工具构成了 Agent 的完整动作空间，使其能够根据用户需求自主选择 and 组合工具完成复杂任务。

4.2.3.3 导航子系统

导航子系统负责处理机器人的路径规划和实时导航，通过集成百度地图 API 实现了精确的步行路线规划。系统采用基于 GPS 的全局定位与基于深度相机的局部避障相结合的导航策略，确保机器人能够安全、准确地引导新生到达目的地。导航信息通过 WebSocket 实时推送到管理端，实现了多机器人位置的实时监控和可视化展示。

4.2.3.4 管理子系统

管理子系统基于成熟的 RuoYi 框架改造而成，采用 Spring Boot + Vue 的技术栈实现前后端分离。系统提供了完善的机器人管理、新生信息管理、志愿者管理等功能模块，通过直观的 Web 界面实现了对整个系统的集中管控。管理端还集成了实时的机器人状态监控和任务调度功能，支持远程下发指令和角色切换，极大地提高了系统的运维效率。

4.2.4 系统技术选型

系统的技术选型充分考虑了各子系统的特点和需求。各子系统主要技术选型及考虑因素如下：

4.2.4.1 机器人子系统技术选型

机器人端采用 ROS (Robot Operating System) 作为基础框架，利用其成熟的硬件抽象层和丰富的功能包生态，配合 Flask 轻量级 Web 框架提供 HTTP 服务接口。

4.2.4.2 智能体子系统技术选型

智能体子系统选用 Python 作为主要开发语言，充分利用其在 AI 领域的生态优势，通过 Dify 平台实现 Agent 能力的快速集成和部署。

4.2.4.3 管理子系统技术选型

管理端采用企业级的技术栈，后端基于 Spring Boot 框架构建微服务架构，使用 Spring Security 实现安全认证，MyBatis 作为持久层框架，JWT 实现无状态的用户认证。前端采用 Vue.js 框架开发单页应用，提供响应式的用户界面。数据存储方面，使用 MySQL 作为主数据库存储结构化数据，Redis 作为缓存中间件提升系统性能。

4.2.4.4 导航子系统技术选型

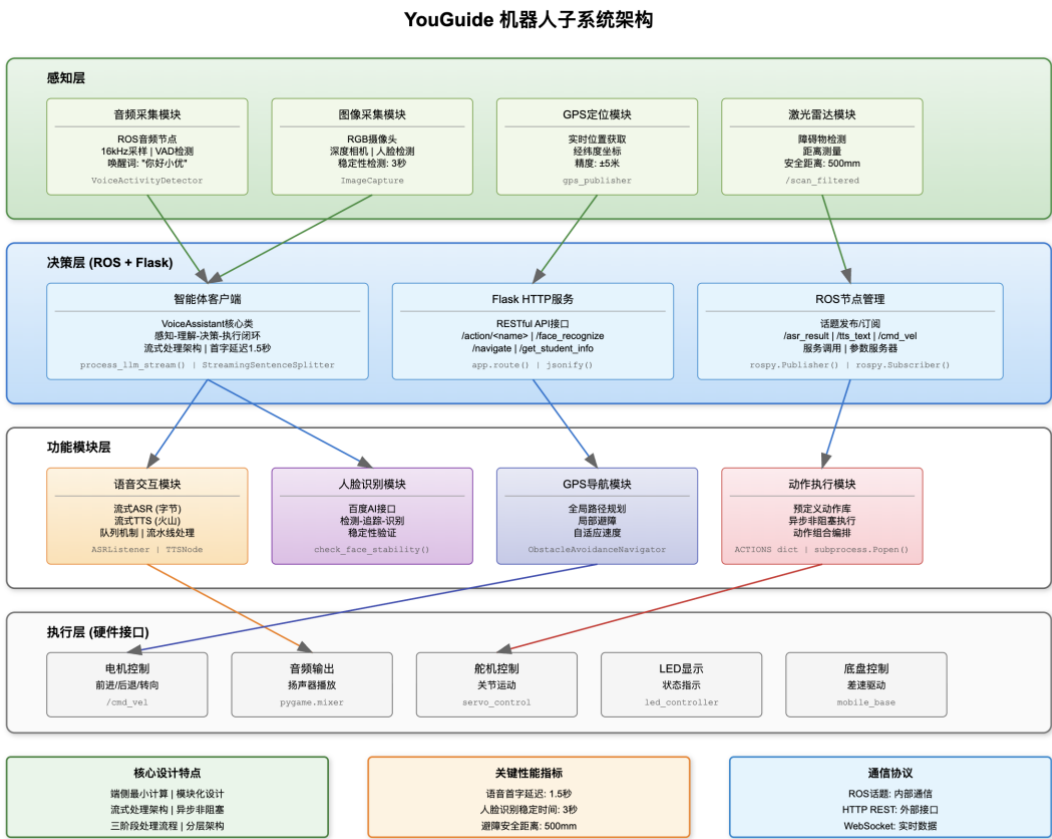
导航子系统在技术选型上注重实时性和可靠性，后端采用 Flask 框架快速构建 API 服务，前端使用 Vue.js 开发实时地图展示界面，通过 WebSocket 协议实现服务器与客户端的双向实时通信。整体架构设计确保了系统的高可用性、可扩展性和易维护性，为后续的功能迭代和性能优化奠定了坚实基础。

4.3 子系统设计

4.3.1 机器人子系统设计

4.3.1.1 子系统模块划分与整合

机器人子系统作为 YouGuide 系统的感知和执行层，承担着与新生直接交互的重要职责。系统采用分层架构设计，将复杂的机器人功能解耦为感知层、决策层和执行层三个层次。感知层包括语音采集、图像捕获和传感器数据处理；决策层负责意图理解和行为规划；执行层实现具体的动作控制和反馈输出。



在迎新场景中，机器人需要应对嘈杂环境下的语音交互、动态人流中的身份识别、复杂地形下的导航避障等挑战。系统通过 ROS 的分布式架构，将各功能模块封装为独立节点，通过话题机制实现数据流转，通过服务机制提供同步调用。这种设计不仅保

证了系统的实时性和可靠性，还为后续功能扩展预留了接口。

4.3.1.2 智能体客户端设计

智能体客户端是连接机器人硬件与 AI 能力的桥梁，其核心设计理念是实现"感知-理解-决策-执行"的完整闭环。在迎新场景下，新生可能会提出各种复杂的问题，如"我是计算机专业的，宿舍在哪里？"、"我还需要办理哪些手续？"等。客户端需要协调多个模块完成理解和响应。

```
Python
class VoiceAssistant:
    def __init__(self, llm_api_url: str, llm_api_key: str):
        # 初始化各个功能组件
        self.llm_client = LLMClient(llm_api_key, llm_api_url)
        self.asr_listener = ASRListener()
        self.image_capture = ImageCapture()

        # 流式处理组件
        self.text_queue = asyncio.Queue()
        self.splitter = StreamingSentenceSplitter()

    async def process_llm_stream(self, query: str, image_path:
Optional[str] = None):
        """处理 LLM 流式响应并分句"""
        try:
            for chunk in self.llm_client.stream_chat(query,
image_path):
                # 流式分句处理
                sentences = self.splitter.process_chunk(chunk)
                for sentence in sentences:
                    if sentence.strip():
                        await self.text_queue.put(sentence)
                        # 立即触发 TTS，减少延迟
                        await asyncio.sleep(0.001)
        finally:
            self.llm_done = True
            await self.text_queue.put(None)
```

为了增加交互体验，系统采用了流式处理架构，将 LLM 的输出实时分句并送入 TTS 队列，大幅降低了首字延迟。在实际测试中，用户提问后平均 1.5 秒即可听到机器人开始回答，相比传统的"等待-生成-合成-播放"模式，用户体验提升明显。

4.3.1.3 语音交互模块设计

语音交互模块是机器人与新生交流的主要通道，需要在校园的嘈杂环境中准确识别语音并自然地进行回复。系统采用了“唤醒词触发+连续识别”的交互模式，既保证了隐私安全，又提供了流畅的对话体验。

ASR 节点集成了先进的语音活动检测（VAD）算法，通过能量阈值和静音时长的双重判断，准确识别语音结束时机：

```
Python
class VoiceActivityDetector:
    def __init__(self, sample_rate=16000, energy_threshold=0.01,
silence_duration=3.0):
        self.energy_threshold = energy_threshold
        self.silence_duration = silence_duration
        self.silence_frames = int(silence_duration / frame_length)
        self.recent_frames = deque(maxlen=self.silence_frames)
        self.has_spoken = False # 标记是否已经检测到过语音

    def add_audio_data(self, audio_data):
        """添加音频数据并检测语音活动"""
        energy = np.mean(np.abs(audio_array.astype(np.float32))) /
32768.0
        has_voice = energy > self.energy_threshold
        self.recent_frames.append(has_voice)

        if has_voice:
            self.is_speaking = True
            self.has_spoken = True
        elif self.has_spoken and not any(self.recent_frames):
            # 只有说过话后的静音才触发结束
            self.is_speaking = False
```

TTS 模块采用火山引擎的神经网络语音合成技术，支持多种音色和语速调节。系统通过队列机制实现了生成和播放的流水线处理，当一句话在合成时，前一句已经在播放，实现了真正的流式语音输出。在迎新场景中，机器人可以根据不同情况调整语音风格，如对待紧张的新生使用更温和的语调。

4.3.1.4 动作交互模块设计

动作交互是提升机器人亲和力的重要手段。在迎新场景中，恰当的肢体语言能够缓解新生的紧张情绪，增强交互的自然性。系统设计了一套完整的动作管理框架，支持基础动作的组合和情境化触发。

```

Python
ACTIONS = {
    "举手": {
        "cmd_type": "run_node",
        "path":
"/home/lemon/robot_ros_application/catkin_ws/src/ros_actions_node/
scripts/举手.py"
    },
    "鞠躬": {
        "cmd_type": "run_node",
        "path":
"/home/lemon/robot_ros_application/catkin_ws/src/ros_actions_node/
scripts/鞠躬.py"
    },
    # ... 其他动作配置
}

@app.route('/action/<action_name>', methods=['GET'])
def execute_action(action_name):
    """执行指定的动作（异步启动，立即返回）"""
    process = subprocess.Popen(
        ['python3', script_path],
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
        preexec_fn=os.setsid # 创建新进程组，避免阻塞
    )
    return jsonify({
        "success": True,
        "action": action_name,
        "process_id": process.pid,
        "note": "动作正在后台执行，无需等待完成"
    })

```

动作执行采用异步非阻塞模式，确保机器人在执行动作时仍能响应其他请求。系统预定义了迎新场景常用的动作序列，如"欢迎新同学"（举手+点头）、"指引方向"（转身+手势）等，Agent 可以根据对话内容自主选择合适的动作组合。

4.3.1.5 人脸识别模块设计

人脸识别模块在迎新场景中发挥着关键作用，不仅用于身份验证，还能帮助机器人提供个性化服务。系统采用了"检测-追踪-识别"的三阶段处理流程，确保识别的准确

性和用户体验。

考虑到新生可能因为好奇而短暂看向机器人，"稳定性检测"机制的设计要求人脸在指定区域内稳定保持 3 秒才触发识别，有效避免了误识别和隐私问题：

Python

```
def check_face_stability(face, frame_shape):
    """检查人脸是否在判定范围内稳定保持"""
    if not check_face_in_range(face, frame_shape):
        # 人脸不在范围内，重置计时器
        Face.face_stable_start_time = 0
        Face.face_is_stable = False
        return False

    # 人脸在判定范围内
    if Face.face_stable_start_time == 0:
        Face.face_stable_start_time = time.time()
    elif time.time() - Face.face_stable_start_time >=
Face.face_stable_duration:
        Face.face_is_stable = True
        if not Face.recognition_triggered:
            Face.recognition_triggered = True
        return True
    return False
```

识别成功后，机器人可以主动问候："你好，王同学，你的宿舍是雁南 S5 楼 301 室。"

4.3.1.6 GPS 导航模块设计

由于机器人平台限制，本模块在实验室环境中使用模拟 GPS 信号进行功能验证。系统设计了完整的 GPS 导航框架，为未来实际部署预留了接口。GPS 导航模块专门针对校园环境的特点进行了优化设计。北邮沙河校区地形复杂，既有开阔的广场，也有狭窄的小径，还有台阶、斜坡等特殊地形。系统采用了全局路径规划与局部避障相结合的导航策略。

Python

```
class ObstacleAvoidanceNavigator:
    def avoid_obstacle(self, mean_distance):
        """分级避障策略"""
        if mean_distance < 150:
            rospy.loginfo("Too close to detect, stopping")
```

```

        return "stop"
    elif mean_distance < CLOSE_DISTANCE: # 250mm
        rospy.loginfo("Too close, moving backward")
        self.slow_walk("backward", 1)
        return "backward"
    elif mean_distance < SAFE_DISTANCE: # 500mm
        rospy.loginfo("Obstacle detected, turning right 30
degrees")
        self.slow_walk("rotation", angle=-30)
        return "turn"
    elif mean_distance > FAR_DISTANCE: # 1000mm
        rospy.loginfo("Path clear, moving forward fast")
        self.slow_walk("forward", 4)
        return "forward_fast"

```

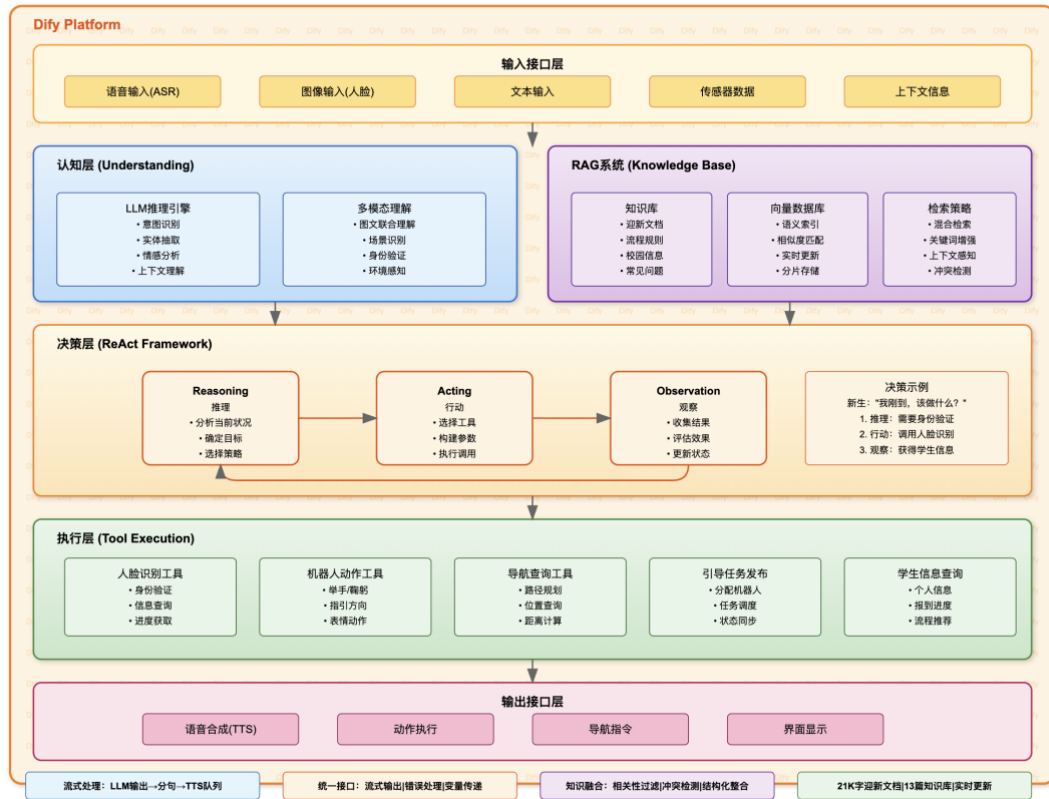
导航系统实现了自适应速度控制：在人流密集区域降低速度确保安全，在空旷区域加速提高效率。同时，系统会实时计算剩余距离和预计到达时间，让新生对行程有清晰的预期。

4.3.2 智能体系统设计

4.3.2.1 子系统模块划分与整合

智能体系统是 YouGuide 的"大脑"，负责理解用户意图、规划行动方案、调用合适工具完成任务。系统基于 Dify 平台构建，充分利用其 workflow 编排能力和插件扩展机制。在架构上，智能体系统分为三个核心层次：认知层（基于 LLM 的语言理解和推理）、决策层（基于 ReAct 的行动规划）、执行层（工具调用和结果整合）。

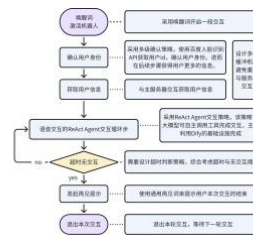
YouGuide 智能体子系统架构 (基于Dify平台)

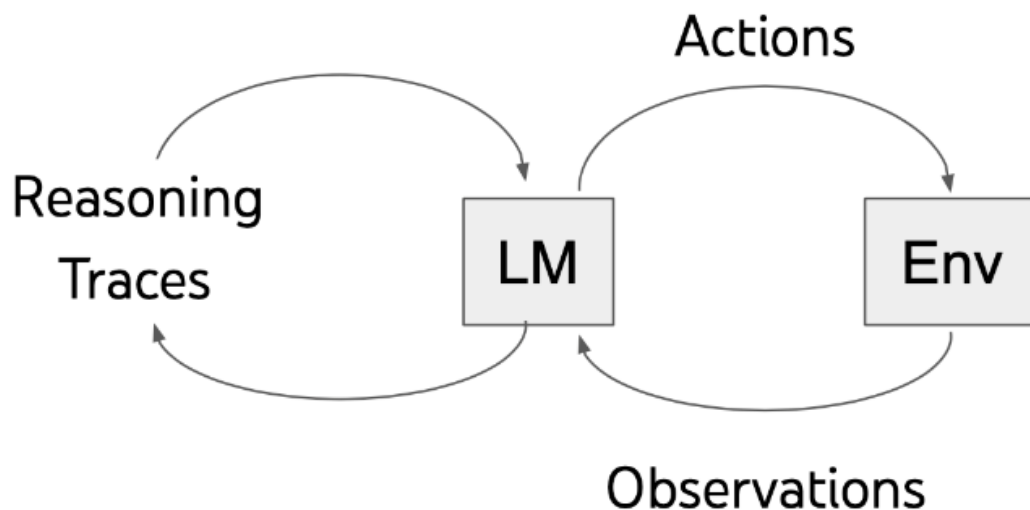


在迎新场景中，智能体需要处理的任务复杂多样：从简单的问路（“食堂在哪里？”）到复杂的流程咨询（“我是国际学生，需要办理哪些特殊手续？”），从情感支持（“我有点紧张，第一次离家这么远”）到紧急处理（“我的行李丢了怎么办？”）。系统通过模块化设计，将不同类型的能力封装为独立的工具，由 Agent 根据场景灵活组合使用。

4.3.2.2 ReAct 架构与工具调用集成设计

ReAct (Reasoning and Acting) 是一种将 LLM 的推理能力与实际行动相结合的架构范式。传统的 LLM 只能生成文本回复，而 ReAct 使模型能够：1) 推理当前情况；2) 决定需要采取的行动；3) 执行行动并观察结果；4) 基于结果继续推理。这种“思考-行动-观察”的循环使得 Agent 能够完成复杂的多步骤任务。





ReAct (Reason + Act)

在 YouGuide 系统中，ReAct 架构的实现充分考虑了迎新场景的特点。以一个典型的交互为例：

新生："我刚到学校，不知道接下来该做什么。"

Agent 的 ReAct 处理流程：

1. **Reasoning (推理)**：用户是新到校的学生，需要了解报到流程
2. **Acting (行动)**：调用人脸识别工具获取学生身份
3. **Observation (观察)**：识别结果显示是计算机学院的张同学，已完成宿舍登记
4. **Reasoning (再次推理)**：基于学生的当前进度，下一步应该是领取资料
5. **Acting (再次行动)**：查询资料领取点位置，发布引导任务

系统的工具设计遵循统一的接口规范：

```

Python
class FaceRecognitionStudentTool(Tool):
    def _invoke(self, tool_parameters: dict[str, Any]) ->
Generator[ToolInvokeMessage, None, None]:
    """执行人脸识别并获取学生信息"""
    try:
        # 步骤 1：调用人脸识别服务

        yield self.create_text_message("正在进行人脸识别...")

        face_recognition_url =
  
```

```

self.runtime.credentials.get('face_recognition_url')
    student_id = recognize_face(face_recognition_url)

    if not student_id:
        yield self.create_text_message("未能识别到学生身份")
        yield self.create_json_message({"status": "error",
"message": "人脸识别失败"})
        return

    yield self.create_text_message(f"人脸识别成功，学生 ID:
{student_id}")

    # 步骤 2：查询学生详细信息

    yield self.create_text_message("正在查询学生信息...")

    student_info_url =
self.runtime.credentials.get('student_info_url')
    student_info = get_student_info(student_info_url,
student_id)

    # 步骤 3：返回结构化结果
    yield self.create_json_message({
        "student_id": student_id,
        "student_info": student_info,
        "status": "success"
    })

    # 设置输出变量供后续工具使用
    yield self.create_variable_message("student_id",
str(student_id))
    yield self.create_variable_message("student_info",
str(student_info))

    except Exception as e:
        yield self.create_text_message(f"工具执行错误：
{str(e)}")
        yield self.create_json_message({"status": "error",
"message": str(e)})

```

每个工具的设计都考虑了实际使用场景的需求：

1. **流式输出**：工具执行过程中实时反馈进度，让用户了解系统正在做什么
2. **错误处理**：完善的异常捕获和错误信息，便于问题定位
3. **变量传递**：通过 `create_variable_message` 在工具间传递数据
4. **结构化输出**：返回 JSON 格式的结果，便于 Agent 解析和使用

特别值得一提的是引导任务发布工具，它实现了答疑机器人与引导机器人的协同：

```
Python
class PublishGuideTaskTool(Tool):
    def _invoke(self, tool_parameters: dict[str, Any]) ->
Generator[ToolInvokeMessage, None, None]:
    """发布引导任务到最近的引导机器人"""
    destination = tool_parameters.get("destination",
    "").strip()

    # 验证目的地合法性
    if destination not in self.PREDEFINED_DESTINATIONS:
        yield self.create_text_message(f"错误：无效的目的地。")
        return

    # 构建请求
    request_data = {
        "robot_id": self.ROBOT_ID,
        "destination": destination
    }

    # 调用 API 分配最近的引导机器人
    response = requests.post(url, json=request_data)

    if response_data.get("success", False):
        task_details = response_data.get("task_details", {})
        success_message = (
            f"引导任务发布成功！\n"
            f"目的地：{task_details.get('destination',
destination)}\n"
            f"分配的引导机器人 ID：
{task_details.get('guide_robot_id', 'N/A')}\n"
            f"距离引导机器人：
{task_details.get('distance_to_guide', 'N/A')} 米"
```

```
)  
yield self.create_text_message(success_message)
```

4.3.2.3 RAG 系统设计

RAG (Retrieval-Augmented Generation) 技术是解决 LLM 幻觉问题的关键方案。在迎新场景中，准确性至关重要——错误的宿舍号、过时的流程信息都可能给新生带来困扰。YouGuide 的 RAG 系统采用了多层次的知识组织架构。

知识库组织结构：

1. **静态知识库**：包含学校地图、建筑物信息、院系设置等相对稳定的信息
2. **动态知识库**：包含最新的报到流程、临时通知、实时人流情况等
3. **个性化知识库**：针对不同学院、不同类型学生的特殊要求

检索策略优化：系统采用混合检索策略，结合了语义相似度和关键词匹配：

- **语义检索**：使用预训练的句子嵌入模型，计算问题与文档片段的语义相似度
- **关键词增强**：识别问题中的关键实体（如"计算机学院"、"S5 宿舍楼"），优先检索包含这些实体的文档
- **上下文感知**：考虑对话历史，如果用户之前提到自己是"人工智能专业"，后续检索会优先返回相关内容

知识融合机制：RAG 系统不是简单地将检索结果拼接到 prompt 中，而是进行智能融合：

1. **相关性过滤**：只保留相关度超过阈值的文档片段
2. **冲突检测**：当多个文档包含矛盾信息时，优先采用更新时间较近的
3. **结构化整合**：将检索结果组织成结构化格式，便于 LLM 理解和使用

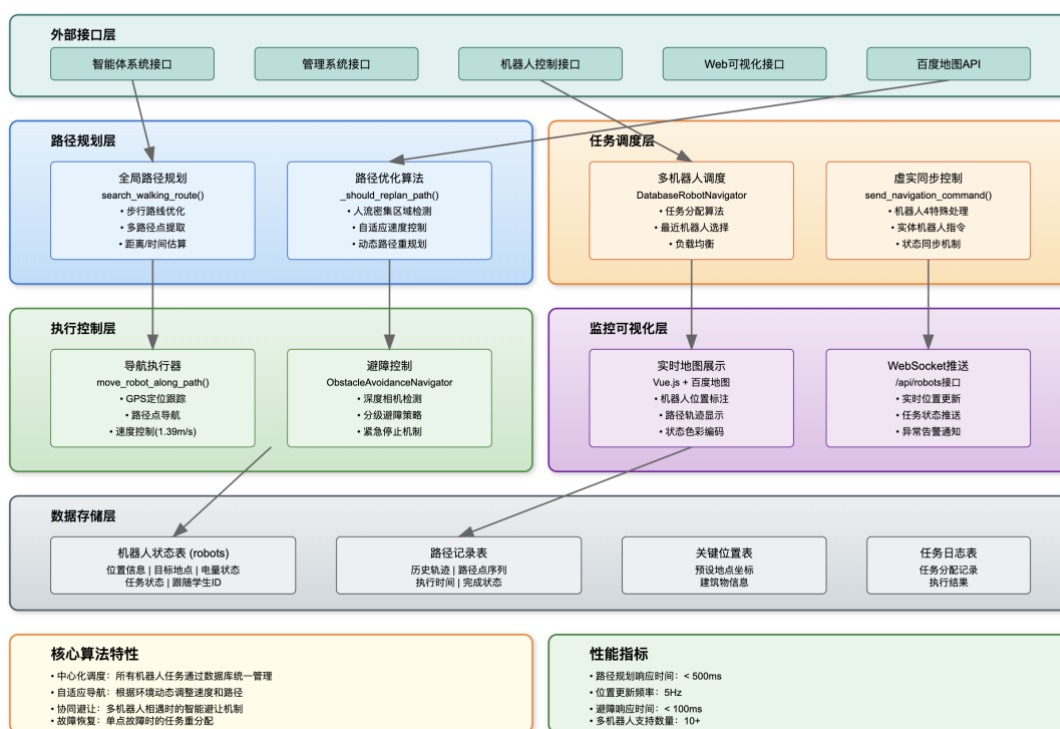
本系统爬取了北京邮电大学迎新网上的 13 篇文档，共 21K 字符，为 Agent 文档打下了坚实基础。

4.3.3 导航系统设计

4.3.3.1 子系统模块划分与整合

导航系统是实现机器人自主移动和多机协同的核心子系统。考虑到迎新场景人流密集、环境动态变化的特点，系统采用了分层导航架构：全局路径规划层负责宏观路线设计，局部避障层处理实时障碍物，任务调度层协调多个机器人的任务分配，监控可视化层提供全局态势感知。

导航子系统架构图



系统的模块化设计确保了各层既能独立工作，又能协同配合。例如，当引导机器人带领新生前往宿舍时，全局规划层会选择人流较少的路线，局部避障层会实时调整以避开临时障碍物，任务调度层会通知其他机器人避让，监控层会实时显示机器人位置供管理员监督。

4.3.3.2 多机器人导航模块设计

多机器人协同导航是 YouGuide 系统的一大亮点。在迎新高峰期，多个机器人需要同时工作，既要避免相互干扰，又要实现任务的合理分配。系统通过数据库驱动的中心化调度机制，实现了高效的多机协同。

核心的导航控制逻辑通过 `DatabaseRobotNavigator` 类实现：

Python

```
class DatabaseRobotNavigator:
    def __init__(self, ak: str, db_config: dict):
        self.ak = ak # 百度地图 API 密钥
        self.db_config = db_config
        self.robot_paths = {} # 存储每个机器人的路径信息
        self.robot_4_first_update = {} # 虚实同步标记

    def get_robots_with_targets(self) -> List[Dict]:
```

```

        """获取所有有目标位置的机器人"""
        query = """
        SELECT robot_id, current_latitude, current_longitude,
               target_latitude, target_longitude
        FROM robots
        WHERE target_latitude IS NOT NULL
              AND target_longitude IS NOT NULL
              AND health_state_id = 1  -- 只调度健康的机器人
              AND robot_type_id = 0    -- 只调度引导型机器人
        """
        self.cursor.execute(query)
        robots = self.cursor.fetchall()

        # 检测新任务并触发虚实同步
        for robot in robots:
            if robot['robot_id'] == 4 and robot['robot_id'] not in self.robot_paths:
                # 机器人 4 是实体机器人，需要同步控制

        self.send_navigation_command((robot['target_latitude'],
                                       robot['target_longitude']))

```

系统的路径规划采用了自适应算法，根据当前场景动态调整：

```

Python
def move_robot_along_path(self, robot_id: int, current_pos:
    Tuple[float, float],
                        target_pos: Tuple[float, float], speed: float
    = 1.39):
    """沿路径移动机器人"""

    # 检查是否需要重新规划路径
    if self._should_replan_path(robot_id, target_pos):
        route_data = self.search_walking_route(current_pos,
        target_pos)
        path_points = self.extract_path_points(route_data)
        self.robot_paths[robot_id] = {
            'target': target_pos,
            'path_points': path_points,
            'current_point_index': 0
        }

```

```

# 计算到下一个路径点的移动
next_point = self._get_next_waypoint(robot_id)

# 自适应速度控制
if self._is_crowded_area(current_pos):
    speed *= 0.5 # 人流密集区域减速
elif self._is_open_area(current_pos):
    speed *= 1.5 # 空旷区域加速

# 执行移动
self._execute_movement(robot_id, current_pos, next_point,
speed)

```

值得一提的是虚实机器人同步机制。系统中的机器人 4 对应实体机器人，当数据库中机器人 4 被分配任务时，系统会自动向实体机器人发送导航指令：

```

Python
def send_navigation_command(self, target_gps: Tuple[float, float])
-> bool:
    """向实体机器人发送导航命令"""
    url = f"http://{self.robot_ip}:{self.robot_port}/navigate"
    data = {
        "target_gps": [target_gps[1], target_gps[0]] # [经度, 纬
度]
    }

    response = requests.post(url, json=data, timeout=5)
    if response.status_code == 200:
        result = response.json()
        logger.info(f"实体机器人导航命令已发送：距离
{result.get('distance_meters')}米")
        return True
    return False

```

4.3.3.3 导航信息可视化模块设计

可视化模块为管理人员提供了直观的全局态势感知能力。在迎新现场，管理人员需要实时了解所有机器人的位置、状态和任务执行情况，以便及时调度和干预。



系统通过 Web 界面展示实时地图，集成了多层信息展示：

Python

```
@app.route('/api/robots', methods=['GET'])
def get_robots():
    """获取所有机器人信息"""
    query = """
        SELECT r.*,
               kp.name as target_name,
               hs.name as health_state,
               rt.name as robot_type
        FROM robots r
        LEFT JOIN key_positions kp ON r.target_position_id = kp.id
        LEFT JOIN health_states hs ON r.health_state_id = hs.id
        LEFT JOIN robot_types rt ON r.robot_type_id = rt.id
    """
    cursor.execute(query)
    robots = cursor.fetchall()

    # 添加实时计算的信息
    for robot in robots:
        # 计算剩余电量可工作时间
        robot['estimated_working_time'] =
        calculate_battery_time(robot['battery_level'])
        # 计算到目标的剩余距离
```

```

        if robot['target_latitude'] and robot['target_longitude']:
            robot['distance_to_target'] = calculate_distance(
                robot['current_latitude'],
                robot['current_longitude'],
                robot['target_latitude'],
                robot['target_longitude']
            )
            # 添加状态标识
            robot['status_color'] =
            get_status_color(robot['health_state_id'], robot['battery_level'])

    return jsonify(robots)

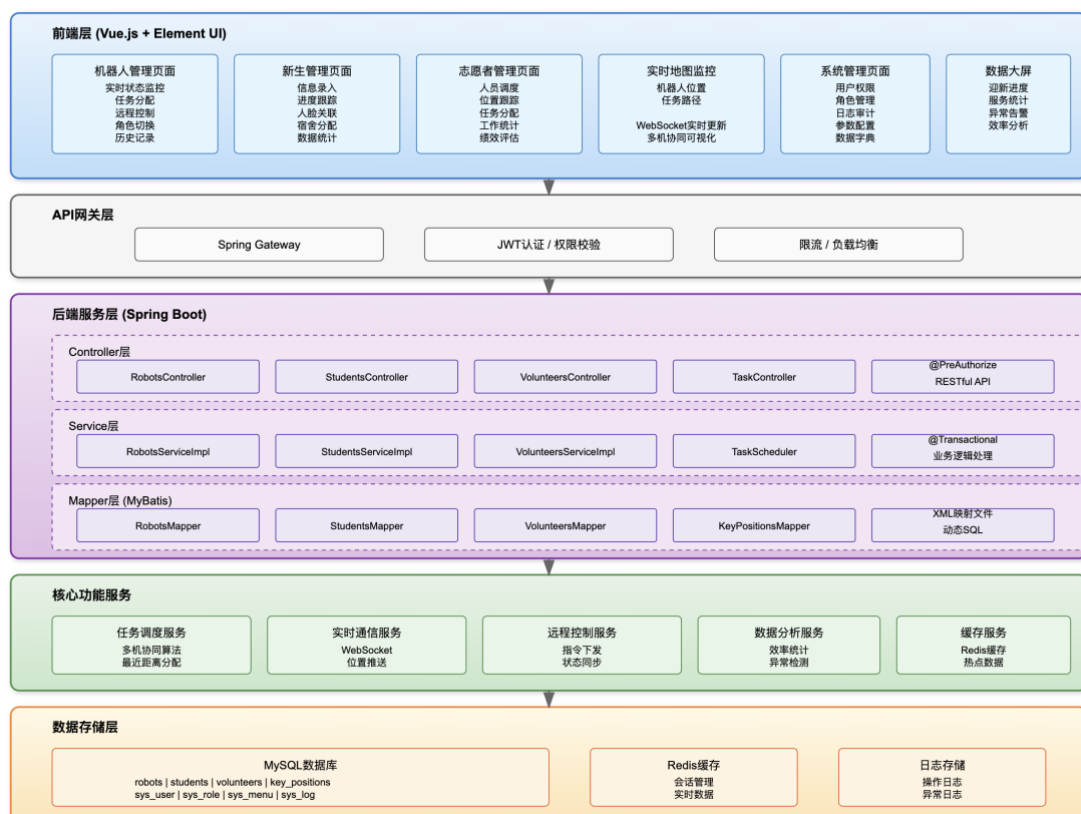
```

4.3.4 管理系统设计

4.3.4.1 子系统模块划分与整合

管理系统基于成熟的 RuoYi 企业级框架构建，继承了其完善的权限管理、日志审计、数据字典等基础功能，并针对迎新场景进行了深度定制。系统采用经典的 MVC 架构，通过 Spring Boot 实现业务逻辑，MyBatis 处理数据持久化，Vue.js 构建响应式前端界面。

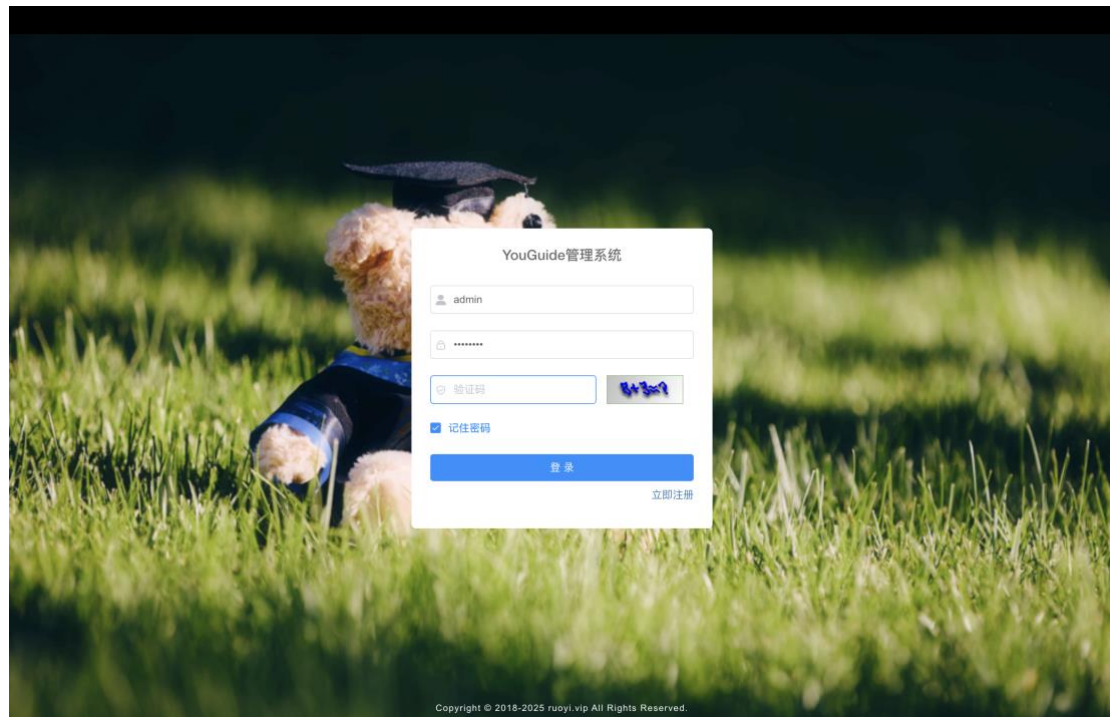
YouGuide 管理子系统架构（基于RuoYi框架）



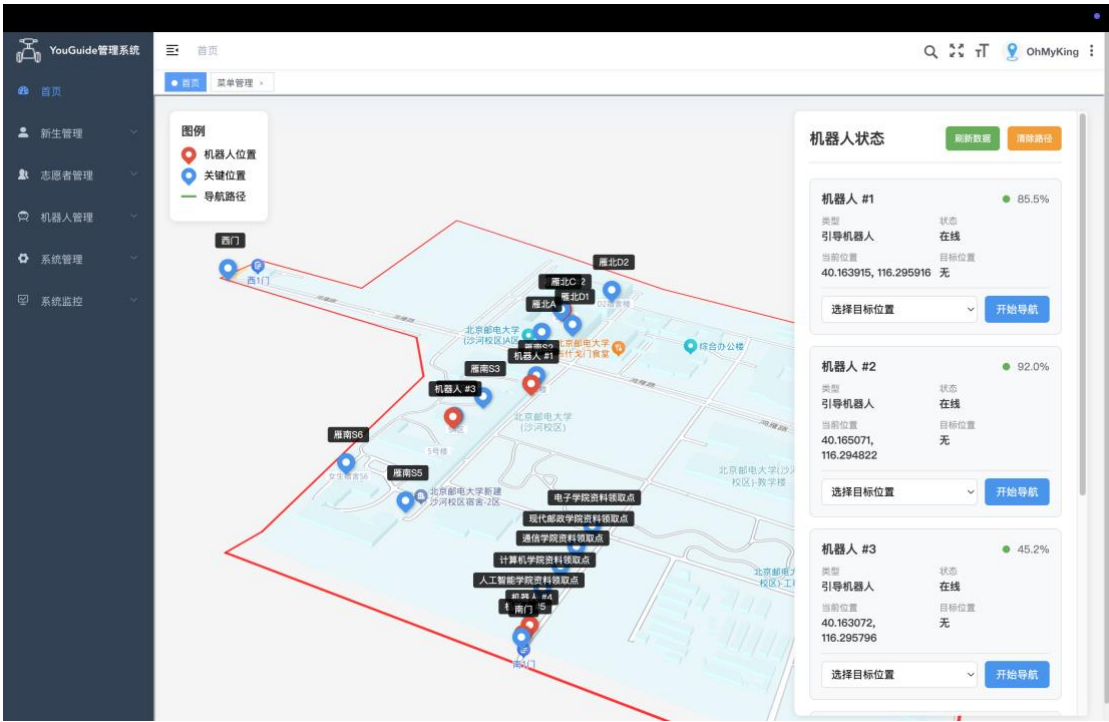
系统的模块划分遵循领域驱动设计（DDD）思想，将迎新业务划分为三个核心领域：机器人管理领域（负责机器人的全生命周期管理）、新生服务领域（处理新生信息和报到流程）、人员协调领域（管理志愿者和工作人员）。每个领域都有独立的领域模型、业务规则和数据访问层。

4.3.4.2 管理系统页面展示

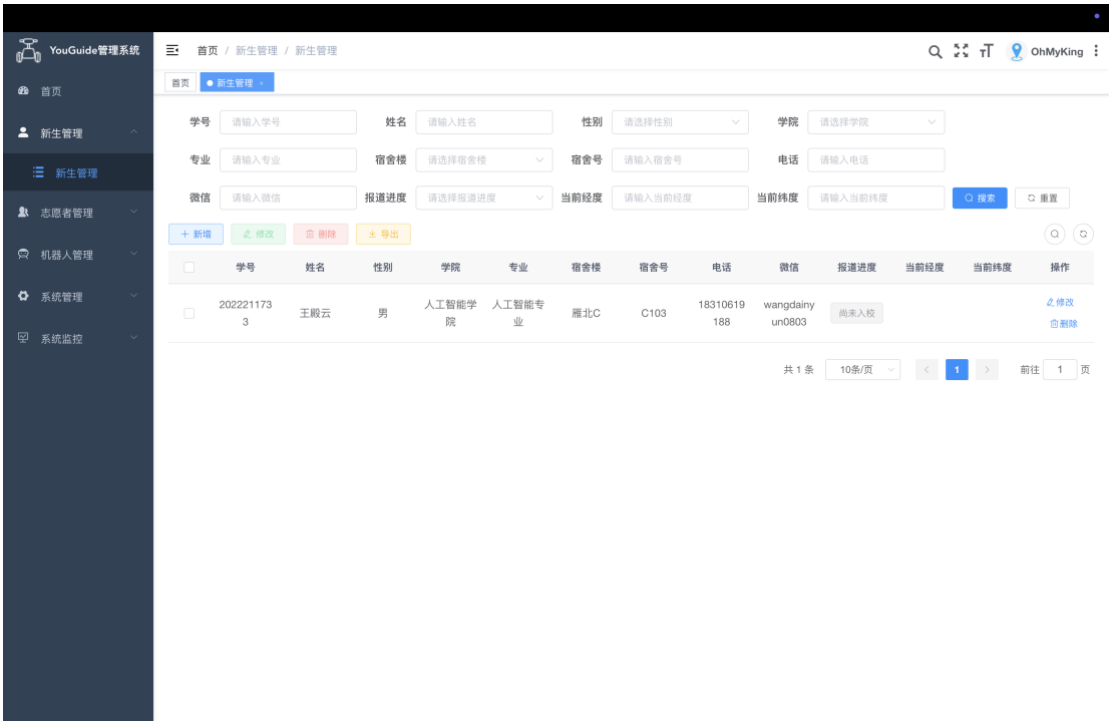
4.3.4.2.1 登录页



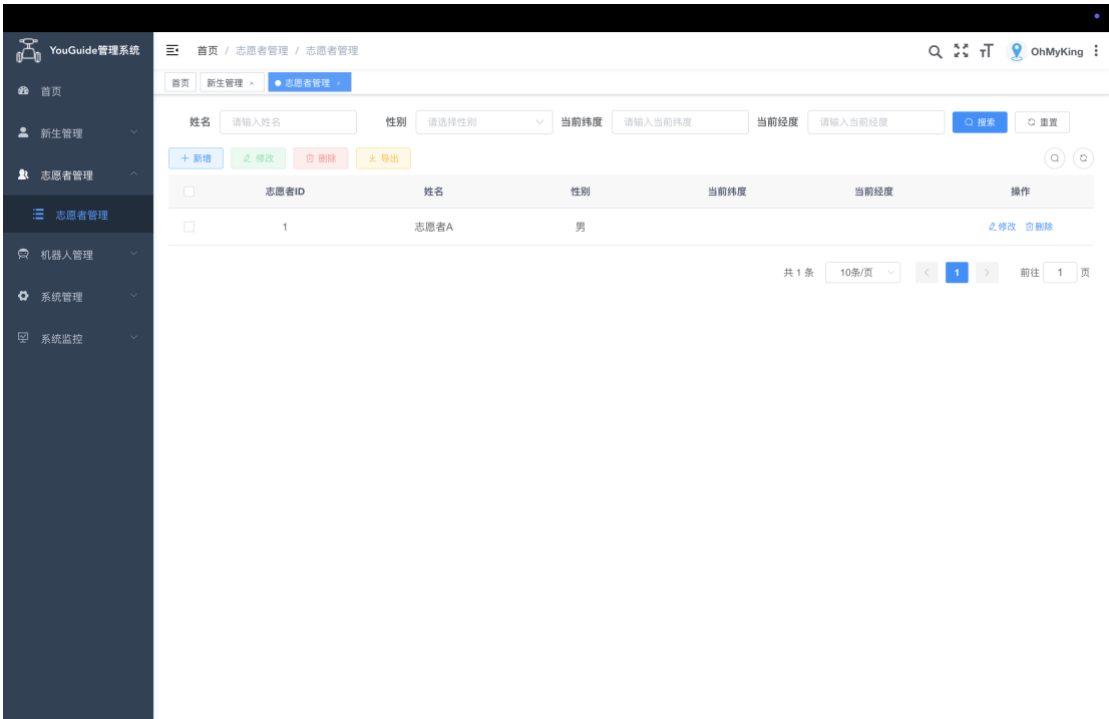
4.3.4.2.2 首页



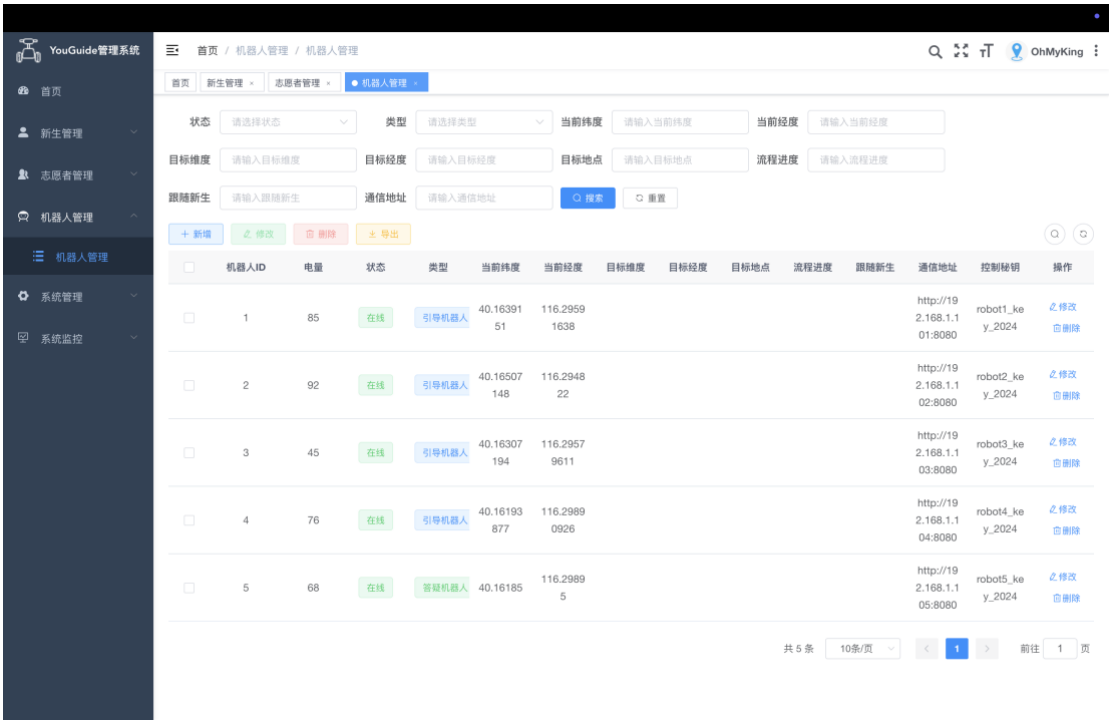
4.3.4.2.3 新生管理页



4.3.4.2.4 志愿者管理页



4.3.4.2.5 机器人管理页



4.3.4.3 机器人管理模块设计

机器人管理模块是整个系统的运营中枢，需要实时监控机器人状态、分配任务、处理异常情况。模块采用了标准的三层架构设计：

实体层设计展现了机器人的完整属性模型：

Java

```
public class Robots extends BaseEntity {
    private static final long serialVersionUID = 1L;

    @Excel(name = "机器人 ID")
    private String robotId;

    @Excel(name = "电量")
    private Long batteryLevel;

    @Excel(name = "状态")
    private Long healthStateId;

    @Excel(name = "类型")
    private Long robotTypeId; // 0:引导机器人, 1:答疑机器人

    @Excel(name = "当前纬度")
    private BigDecimal currentLatitude;

    @Excel(name = "当前经度")
    private BigDecimal currentLongitude;

    @Excel(name = "目标纬度")
    private BigDecimal targetLatitude;

    @Excel(name = "目标经度")
    private BigDecimal targetLongitude;

    @Excel(name = "跟随新生")
    private String followingStudentId;

    @Excel(name = "通信地址")
    private String communicationUrl;

    @Excel(name = "控制密钥")
    private String controlKey;
}
```

服务层实现了复杂的业务逻辑:

```

Java
@Service
public class RobotsServiceImpl implements IRobotsService {
    @Autowired
    private RobotsMapper robotsMapper;

    @Autowired
    private TaskScheduler taskScheduler;

    @Override
    @Transactional
    public int assignNavigationTask(String robotId, String
destination) {
        // 1. 检查机器人状态
        Robots robot =
robotsMapper.selectRobotsByRobotId(robotId);
        if (robot.getHealthStateId() != 1) {
            throw new BusinessException("机器人状态异常, 无法分配任务
");
        }

        // 2. 检查电量是否充足
        double estimatedPower = calculateRequiredPower(
            robot.getCurrentLatitude(),
robot.getCurrentLongitude(),
            destination
        );
        if (robot.getBatteryLevel() < estimatedPower * 1.2) { //
留 20%余量
            throw new BusinessException("机器人电量不足, 请选择其他机
器人");
        }

        // 3. 更新机器人目标位置
        robot.setTargetPositionId(getPositionId(destination));
        robot.setTargetLatitude(getLatitude(destination));
        robot.setTargetLongitude(getLongitude(destination));

        // 4. 发送控制指令
        boolean success = sendNavigationCommand(robot,
destination);
    }
}

```

```

        if (!success) {
            throw new BusinessException("发送导航指令失败");
        }

        // 5. 记录任务日志
        logNavigationTask(robotId, destination);

        return robotsMapper.updateRobots(robot);
    }

    @Override
    public List<Robots> findAvailableGuideRobots(BigDecimal
latitude, BigDecimal longitude) {
        // 查找可用的引导机器人
        Robots query = new Robots();
        query.setRobotTypeId(0L); // 引导机器人
        query.setHealthStateId(1L); // 健康状态
        List<Robots> robots =
robotsMapper.selectRobotsList(query);

        // 按距离排序
        robots.sort((r1, r2) -> {
            double d1 = calculateDistance(latitude, longitude,
r1.getCurrentLatitude(),
r1.getCurrentLongitude());
            double d2 = calculateDistance(latitude, longitude,
r2.getCurrentLatitude(),
r2.getCurrentLongitude());
            return Double.compare(d1, d2);
        });

        // 过滤掉电量不足的机器人
        return robots.stream()
            .filter(r -> r.getBatteryLevel() > 20)
            .collect(Collectors.toList());
    }
}

```

控制层提供 RESTful API 接口：

Java

```

@RestController
@RequestMapping("/system/robots")
public class RobotsController extends BaseController {

    @PreAuthorize("@ss.hasPermi('system:robots:list')")
    @GetMapping("/list")
    public TableDataInfo list(Robots robots) {
        startPage();
        List<Robots> list =
robotsService.selectRobotsList(robots);

        // 增强返回数据，添加实时状态信息
        list.forEach(robot -> {
            robot.putExtra("online",
checkRobotOnline(robot.getCommunicationUrl()));
            robot.putExtra("taskCount",
getTaskCount(robot.getRobotId()));
            robot.putExtra("efficiency",
calculateEfficiency(robot.getRobotId()));
        });

        return getDataTable(list);
    }

    @PostMapping("/switchType/{robotId}")
    @Log(title = "切换机器人类型", businessType =
BusinessType.UPDATE)
    public AjaxResult switchRobotType(@PathVariable String
robotId) {
        Robots robot =
robotsService.selectRobotsByRobotId(robotId);

        // 切换类型：引导机器人 <-> 答疑机器人
        Long newType = robot.getRobotTypeId() == 0L ? 1L : 0L;
        robot.setRobotTypeId(newType);

        // 清除当前任务
        robot.setTargetPositionId(null);
        robot.setTargetLatitude(null);
        robot.setTargetLongitude(null);

        // 发送角色切换指令
    }
}

```

```

        sendRoleChangeCommand(robot, newType);

        return toAjax(robotsService.updateRobots(robot));
    }
}

```

4.3.4.4 新生管理模块设计

新生管理模块负责维护新生的完整信息和报到进度，与人脸识别系统深度集成，实现了"刷脸即知"的智能服务：

```

Java
public class Students extends BaseEntity {
    @Excel(name = "学号")
    private Long studentNum;

    @Excel(name = "姓名")
    private String name;

    @Excel(name = "学院")
    private String college;

    @Excel(name = "专业")
    private String major;

    @Excel(name = "宿舍楼")
    private Long dormitoryId;

    @Excel(name = "宿舍号")
    private String dormitoryNo;

    @Excel(name = "报道进度")
    private Long flowPositionId; // 当前完成到哪个步骤

    @Excel(name = "当前经度")
    private BigDecimal currentLatitude;

    @Excel(name = "当前纬度")
    private BigDecimal currentLongitude;
}

```

服务层实现了报到流程的自动化管理：

```
Java
@Service
public class StudentsServiceImpl implements IStudentsService {

    @Override
    public StudentReportProgress getReportProgress(Long studentId)
    {
        Students student =
studentsMapper.selectStudentsByStudentId(studentId);

        // 获取学生的报到进度
        List<ReportStep> allSteps =
getReportSteps(student.getCollege());
        List<ReportStep> completedSteps =
getCompletedSteps(student.getFlowPositionId());
        List<ReportStep> remainingSteps =
getRemainingSteps(allSteps, completedSteps);

        // 构建进度信息
        StudentReportProgress progress = new
StudentReportProgress();
        progress.setStudent(student);
        progress.setCompletedSteps(completedSteps);
        progress.setRemainingSteps(remainingSteps);

        progress.setCompletionRate(calculateCompletionRate(completedSteps,
allSteps));
        progress.setNextStep(remainingSteps.isEmpty() ? null :
remainingSteps.get(0));

        // 获取下一步的位置信息
        if (progress.getNextStep() != null) {
            KeyPosition position =
getStepLocation(progress.getNextStep());
            progress.setNextStepLocation(position);
            progress.setDistanceToNextStep(calculateDistance(
                student.getCurrentLatitude(),
student.getCurrentLongitude(),
                position.getLatitude(), position.getLongitude()
            ));
        }
    }
}
```

```

        return progress;
    }

    @Override
    @Transactional
    public void updateReportProgress(Long studentId, Long stepId)
    {
        Students student =
studentsMapper.selectStudentsByStudentId(studentId);

        // 更新进度
        student.setFlowPositionId(stepId);
        studentsMapper.updateStudents(student);

        // 发送进度通知
        sendProgressNotification(student, stepId);

        // 如果完成所有流程，触发完成事件
        if (isAllStepsCompleted(student)) {
            handleReportCompletion(student);
        }
    }
}

```

4.3.4.5 志愿者管理模块设计

志愿者管理模块实现了人机协同的管理机制。在期望的交互场景中，迎新现场的志愿者可以通过手机端（待开发）实时查看机器人状态，在必要时进行人工干预：

```

Java
@Service
public class VolunteersServiceImpl implements IVolunteersService {

    @Override
    public void assignEmergencyTask(String volunteerId, String
taskType, String location) {
        Volunteers volunteer =
volunteersMapper.selectVolunteersByVolunteerId(volunteerId);

        // 创建紧急任务
        EmergencyTask task = new EmergencyTask();
    }
}

```

```

        task.setVolunteerId(volunteerId);
        task.setTaskType(taskType);
        task.setLocation(location);
        task.setPriority(calculatePriority(taskType));
        task.setCreateTime(new Date());

        // 发送任务通知
        sendTaskNotification(volunteer, task);

        // 更新志愿者位置到任务地点

volunteer.setCurrentLatitude(getLocationLatitude(location));

volunteer.setCurrentLongitude(getLocationLongitude(location));
        volunteersMapper.updateVolunteers(volunteer);

        // 记录任务分配日志
        logTaskAssignment(volunteer, task);
    }

    @Override
    public List<NearbyResource> getNearbyResources(BigDecimal
latitude, BigDecimal longitude) {
        List<NearbyResource> resources = new ArrayList<>();

        // 获取附近的机器人
        List<Robots> nearbyRobots =
robotsService.findRobotsWithinRadius(latitude, longitude, 500);
        resources.addAll(nearbyRobots.stream()
            .map(r -> new NearbyResource("robot", r.getRobotId(),
                calculateDistance(latitude, longitude,
r.getCurrentLatitude(), r.getCurrentLongitude()))
            .collect(Collectors.toList()));

        // 获取附近的志愿者
        List<Volunteers> nearbyVolunteers =
findVolunteersWithinRadius(latitude, longitude, 500);
        resources.addAll(nearbyVolunteers.stream()
            .map(v -> new NearbyResource("volunteer",
v.getVolunteerId(),
                calculateDistance(latitude, longitude,
v.getCurrentLatitude(), v.getCurrentLongitude()))

```

```
        .collect(Collectors.toList()));

        // 按距离排序

resources.sort(Comparator.comparing(NearbyResource::getDistance));

        return resources;
    }
}
```

管理系统通过这种精细化的设计，不仅实现了对机器人、新生、志愿者的全面管理，还通过数据驱动的方式不断优化服务质量。系统记录的每一次交互、每一个任务的执行情况，都会成为改进算法、优化流程的宝贵数据，真正实现了智慧化的迎新服务。

5. 实验平台说明

5.1 硬件平台

本项目采用 ROBAN 人形机器人作为实验验证平台。该机器人具备以下硬件特性：

基础参数：

- 产品型号：ROBAN
- 外形尺寸：68.68cm × 31.35cm × 18.56cm
- 整机重量：6.60kg
- 机身材质：6 系铝合金框架，PC-ABS 外壳
- 自由度：22 个
- 行走速度：7cm/步，0.6s 一步

计算平台：

- 内置主机：英特尔 NUC8i3BEH
- 处理器：第八代智能英特尔酷睿 i3-8109U
- 内存：DDR4 SO-DIMM 8GB（最高可扩展 32G）
- 存储：120G SSD

感知系统：

- 摄像头：双摄像头系统，包括一个高清摄像头和一个结构光深度摄像头
- 麦克风：头部 6mic 阵列，支持声源定位
- 扬声器：8Ω 1.5W

接口配置：

- DC 电源口、网口、HDMI 接口
- 磁吸接口、3.5mm 音频接口
- USB 2.0 × 1、USB 3.0 × 3

电源系统：

- 电池容量：4000mAh



5.2 软件与开发环境

实验平台采用以下软件环境配置：

操作系统：

- Linux Ubuntu 16.04LTS 64bit (机器人)
- 机器人操作系统：ROS Kinetic (机器人)
- MacBook Air(13-inch,M3,2024) macOS Sequoia 15.5 (计算端、管理端)

开发环境：

- C/C++ 7.4.0
- Python 2.7/Python 3.5+
- OpenCV（图像处理）
- RealSense SDK（深度相机驱动）
- TensorFlow（深度学习框架）

5.3 平台限制说明

由于实验条件和硬件平台的限制，本项目在以下方面进行了适应性调整：

定位系统：ROBAN 机器人平台不具备内置 GPS 模块，因此在实验室测试中，系统采用模拟 GPS 信号进行导航功能验证。通过软件模拟的方式，验证了导航算法的正确性和系统的功能完整性。

计算资源：基于 i3-8109U 处理器的计算能力，系统采用了分布式架构，将计算密集型任务（如大语言模型推理）部署在远端服务器，机器人端仅负责感知和执行等轻量级任务。

这些平台特性和限制为系统设计提供了重要的约束条件，也推动了系统在架构设计上的创新，实现了端云协同的高效计算资源分配策略。

6. 总结与展望

6.1 项目成果总结

通过本项目的实施，我们取得了以下主要成果：

技术成果方面，系统实现了多项技术创新和突破。在系统架构上，采用分布式设计理念，实现了端云协同的高效计算资源分配；在智能交互上，通过流式处理大幅降低了语音对话延迟；在协同导航上，实现了多机器人的智能任务分配和路径规划；在知识服务上，构建了基于 RAG 的精准问答系统。这些技术成果不仅解决了迎新场景的实际问题，也为其他领域的智能服务提供了参考。

应用成果方面，系统在实验室环境下的测试验证了其技术可行性和功能完整性。通过模拟迎新场景和使用虚拟 GPS 信号，验证了系统的核心功能。预期通过引导机器人和答疑机器人的协同工作，可以有效缓解了迎新高峰期的服务压力，提升新生的报到效率和体验。系统的模块化设计也便于后续的功能扩展和场景迁移。

创新成果方面，项目在多个方面实现了创新突破，包括多机器人角色协同机制、知识驱动的智能交互模式、虚实融合的导航体系等。这些创新不仅体现了技术的先进

性，更重要的是真正解决了实际场景中的痛点问题。

6.2 存在的不足与改进方向

尽管 YouGuide 系统取得了显著成果，但在实际运行中仍存在一些需要改进的地方：

1. **硬件限制**：当前机器人的续航能力、运动速度等硬件性能仍有提升空间，特别是在长时间、高强度的迎新服务中，需要频繁充电影响了服务连续性。
2. **环境适应性**：系统在复杂环境下的鲁棒性有待加强，如在极端天气、网络不稳定等情况下的应急处理机制需要进一步完善。
3. **交互自然度**：虽然实现了基本的语音交互，但在理解方言、处理噪音干扰、识别情绪等方面还有提升空间。
4. **应用与部署**：当前系统主要针对单一校区设计，在大规模机器人集群管理等方面还需要进一步研究。
5. **实际部署验证**：当前系统仅在实验室环境下完成研发和测试，使用模拟 GPS 信号进行功能验证。未来需要在真实场景中部署，接入实际的机器人 GPS 定位系统，进行全面的现场验证。